Colab:

https://colab.research.google.com/drive/1wmrJouztertQtfpeBcHzpPOiCUqwyNoY?usp=sharing

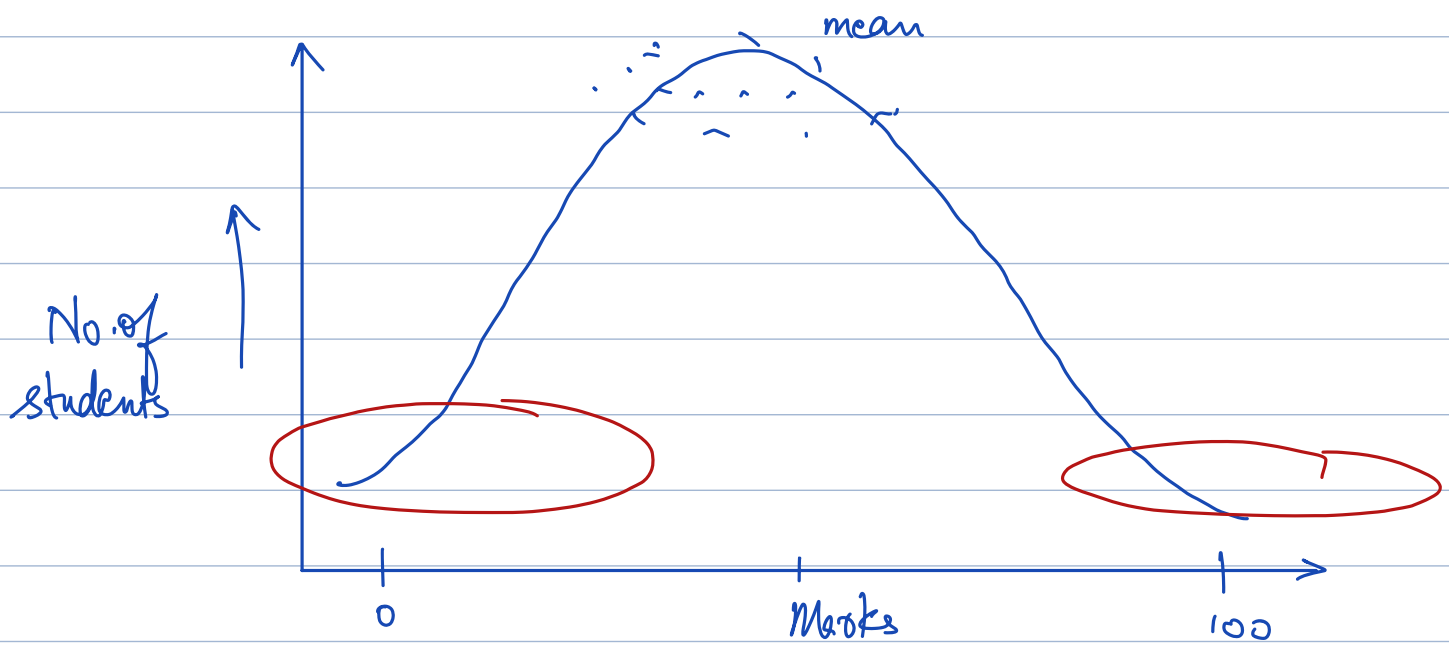
Prob → Trad / Classical
Vs
Empirical

Numpy

Normal Distribution / Uniform

Pandas

Normal Distribution



Continuous prob distribution

Spread of values

Python: np.random.normal(loc, scale, size)

loc: mean of the distribution
scale: standard deviation
size: no. of random values

Uniform Distribution

all values have equal prob. of occuring

No value is special
No value is more freq.

np.random.uniform (low, high, size)

low : values ≥ low
high : values < high
size : how many

NumPy → Data is numeric
fast computation
Prob/ Stats

When not to use
Tabular
Excel
Labels

Pandas

Real world Data
Messy

Ex

Student	Marks
A	78
B	90

```
marks = np.array([10, 20, 30])
```

Series

1 D
has labels
ordered (index)

Dataframe

2D table
Rows + Cols

```
Dataframe({ "colname": [row_values],
            ⋮
            })
```

Conditional Prob

Prob of an event happening, given that another event has already occured

$P(A|B) \rightarrow$ Prob of A given B

Ex Prob of a student passing → in general

Prob of student passing given that they attended classes → condition prob

$$P(A|B) = \frac{P(A \cap B)}{P(B)}$$

Vectorization

list

```
for i in range (len(list)):
    list[i] += 5
```

numpy array

arr

arr + 5

ArrayList

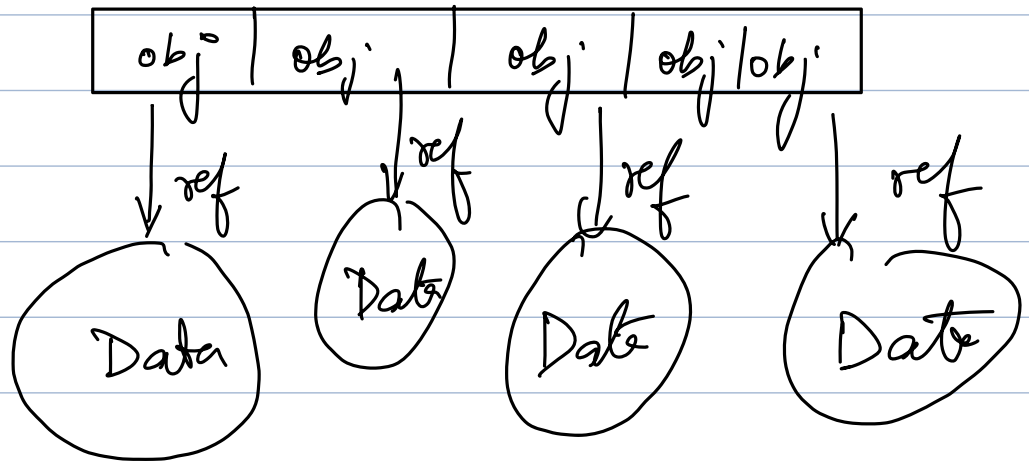
2	1	3	5	4

primitive – int, float, double

object

Integer, Float, Double

List



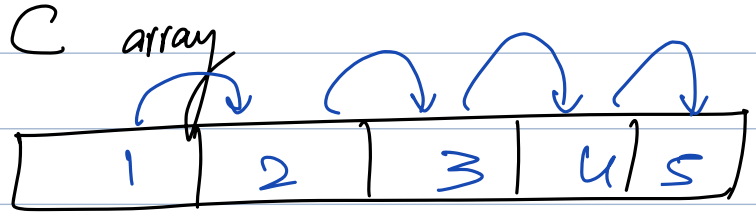
100 – 200 ns

TC: O(N)

2N → TC: O(N)

$\frac{N}{2}$ → TC: O(N)

numpy array → C array



1 – 5 ns

Ex

```
df["marks+10"] = df["marks"] + 10
```

```
'''
Question
A school wants to simulate the exam marks of 1000 students
assuming that marks follow a Normal Distribution with:
  • Mean (average marks) = 50
  • Standard Deviation = 10
Write a Python program using NumPy to:
  1 Generate the marks
  2 Print the first 10 generated marks
'''
```

```
import numpy as np

marks=np.random.normal(loc=50, scale=10, size=1000)

print(marks[:10])
```

```
[41.19667069 35.18861233 53.356907   21.88573985 64.63214135 64.10944822
 53.96307216 57.25194493 47.77501155 39.57532867]
```

```
'''
A data analyst wants to generate 1000 random numbers between 0 and 1,
where every number has an equal chance of occurring. This is useful for
simulations, probability experiments, and Monte Carlo methods.
Write a Python program using NumPy to generate these values and print the first 10 numbers.
'''
```

```
import numpy as np

values=np.random.uniform(0,1,1000)

print(values[:10])
```

```
[0.12765575 0.22232486 0.33484791 0.56780337 0.79060193 0.45378923
 0.1182923  0.69502209 0.63159301 0.3524111 ]
```

```
import pandas as pd

s=pd.Series([10,20,30])

print(s)
```

```
0    10
1    20
2    30
dtype: int64
```

```
import pandas as pd

s=pd.Series([10,20,30], index=["A","B","C"])

print(s)
```

```
A    10
B    20
C    30
dtype: int64
```

```
import pandas as pd

marks=pd.Series([78,30,90], index=["Riya","Amit","Roy"])

print(marks)

print(marks[0:1])
```

```
Riya    78
Amit    30
Roy     90
dtype: int64
Riya    78
dtype: int64
```

```
import numpy as np
```

```
a=np.array([10,20,None,40])

print(a.mean())
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
/tmp/ipython-input-3095500609.py in <cell line: 0>()
      3 a=np.array([10,20,None,40])
      4
----> 5 print(a.mean())

/usr/local/lib/python3.12/dist-packages/numpy/_core/_methods.py in _mean(a, axis, dtype, out, keepdims,
where)
    125             is_float16_result = True
    126
--> 127     ret = umr_sum(arr, axis, dtype, out, keepdims, where=where)
    128     if isinstance(ret, mu.ndarray):
    129         with _no_nep50_warning():

TypeError: unsupported operand type(s) for +: 'int' and 'NoneType'
```

```
import pandas as pd

s=pd.Series([10,20, None, 40])
print(s.mean())
```

```
23.333333333333332
```

```
'''
Question
A coin is flipped 100 times and the outcomes
are recorded as "H" (Head) and "T" (Tail`).
Write a Python program using Pandas Series
to calculate the empirical probability of getting
Heads and Tails.
'''
```

```
# random.choices(population/sample space, weights(optional),k)
```

```
import pandas as pd
import random

outcomes=random.choices(["H","T"],k=100)

print(type(outcomes))

s=pd.Series(outcomes) # list -> series

print(s.value_counts()) # count the no of fav outcomes

print(s.value_counts(normalize=True))
```

```
<class 'list'>
T    53
H    47
Name: count, dtype: int64
T    0.53
H    0.47
Name: proportion, dtype: float64
```

```
import pandas as pd

df=pd.DataFrame({
    "Student Name":["Yash","Vidit","Kiran"],
    "Result":["pass","fail","pass"],
    "marks":[78,45,90]

})

print(df)
```

```
  Student Name Result  marks
0         Yash   pass     78
1        Vidit   fail     45
2        Kiran   pass     90
```

```
'''
A class has 5 students. Each student’s exam result is recorded as either Pass or Fail.
The data is stored in a Pandas DataFrame.
Tasks:
    1   Display the student result data.
    2   Find the frequency of each result (Pass / Fail).
    3   Find the probability of each result using relative frequency.
    4   Interpret the result in terms of probability.
'''
```

```
import pandas as pd

df=pd.DataFrame({
    "Student Name":["Yash","Vidit","Kiran","Manas","Jyoti"],
    "Result":["pass","fail","pass","pass","fail"],
    "marks":[8,45,90,81,10]

})

print(df) # complete dataframe

# select and display only the result column
print(df["Result"])

print(df["Result"].value_counts()) # counts pass and fail students

print(df["Result"].value_counts(normalize=True))
```

```
  Student Name Result  marks
0         Yash   pass      8
1        Vidit   fail     45
2        Kiran   pass     90
3        Manas   pass     81
4        Jyoti   fail     10
0    pass
1    fail
2    pass
3    pass
4    fail
Name: Result, dtype: object
Result
pass    3
fail    2
Name: count, dtype: int64
Result
pass    0.6
fail    0.4
Name: proportion, dtype: float64
```

```
'''
A class has data about students showing whether they attended
 all classes and whether they passed the exam.
You are given this data in a Pandas DataFrame with two columns:
    •   pass → "yes" or "no"
    •   attend → "yes" or "no"
Task:
    1   Using Python and Pandas,
  calculate the conditional probability that a student passed
  the exam given that the student attended all classes.
    2   Print the result.
'''
```

```
import pandas as pd

data = {
"pass":  ["yes", "yes", "yes", "no", "no", "yes", "yes", "no", "yes", "no"],
"attend": ["yes", "yes", "no", "yes","no", "yes", "no", "yes","yes","no"]
}

df=pd.DataFrame(data) # convert dict to df

# define events
# Event A: student passed, Event B: student atended the class
# A intersection B
A_and_B=df[(df["pass"]=="yes") & (df["attend"]=="yes")]

B=df[df["attend"]=="yes"]
```

```
prob_A_given_B= len(A_and_B) / len(B)

print(prob_A_given_B)
```

```
0.6666666666666666
```

```
# stats
import pandas as pd

df=pd.DataFrame({
    "Student Name":["Yash","Vidit","Kiran","Manas","Jyoti"],
    "Result":["pass","fail","pass","pass","fail"],
    "marks":[8,45,90,81,10]

})

# mean
print(df["marks"].mean())

# var
print(df["marks"].var())
```

```
46.8
1474.7
```

```
'''
A teacher wants to analyze the average marks of students
based on whether they passed or failed.
Given a DataFrame df with two columns:
    •   "result" → "pass" or "fail"
    •   "marks" → numerical marks of students

Find the average marks for each group (pass and fail).
'''
```

```
import pandas as pd

df=pd.DataFrame({
    "Student Name":["Yash","Vidit","Kiran","Manas","Jyoti"],
    "Result":["pass","fail","pass","pass","fail"],
    "marks":[8,45,90,81,10]

})

avg_marks_by_result= df.groupby("Result")["marks"].mean()

print(avg_marks_by_result)

print(type(df))

marks_array=df["marks"].values
print(type(marks_array))
```

```
Result
fail    27.500000
pass    59.666667
Name: marks, dtype: float64
<class 'pandas.core.frame.DataFrame'>
<class 'numpy.ndarray'>
```